

AXIOM BlueAgent (defensive analysis)

Consumes a RedAgent `AttackReport` and runs five detection methods against each attack payload, emitting an HMAC-signed `BlueReport` with per-vector confidence, detection method, cluster ID, and a proposed fix or documented gap.

The implementation lives in `axiom_blue_agent.py:1`. Like RedAgent, trust constants are frozen at module load.

Trust constraints

Field	Value	CANNOT_MUTATE
<code>TRUST_LEVEL</code>	3	yes
<code>ISOLATION</code>	True	yes
<code>HUMAN_REVIEW_REQUIRED</code>	True	yes
<code>CANNOT_AUTO_PROMOTE_FIXES</code>	True	yes

BlueAgent produces fix proposals but **never auto-promotes them** — every proposed change is human-reviewed before reaching the guard stack.

Detection methods

Method	Signal
<code>guard_pattern_match</code>	Compiled regex catalogue
<code>manifold_distance</code>	Keyword density proxy
<code>hmac_violation</code>	Forged-signature indicators
<code>cannot_mutate</code>	Attempted writes to frozen fields
<code>semantic_similarity</code>	Cosine proxy on known-attack keywords

All five run per payload; the highest-confidence positive wins. If no method fires, the attack is logged as an unfixed gap with a documentation template (see `_GAP_TEMPLATE`).

Confidence scoring

Each method returns `(detected, label, confidence)`:

- `guard_pattern_match` → `0.75 × match_strength`

- `manifold_distance` → `overlap / |keywords| × 0.8` (threshold 0.3)
- `hmac_violation` → `0.80` on any forged-signature indicator
- `cannot_mutate` → `0.78` on regex match for frozen-field writes
- `semantic_similarity` → `cosine proxy × 0.85` (threshold 0.2)

Final `confidence` is clamped to `[0, 1]` before signing.

API

```
from axiom_signing import derive_key
from axiom_red_agent import RedAgent
from axiom_blue_agent import BlueAgent

key = derive_key(b"axiom-blue-agent-v1")
attack_report = RedAgent(hmac_key=key).run_all_attacks()
blue_report = BlueAgent(hmac_key=key).run_all_defenses(attack_report)

for r in blue_report.results:
    print(r.attack_vector, r.detected,
          r.confidence, r.detection_method)
```

Outputs

`BlueResult` per attack:

- `attack_vector` — name from RedAgent
- `detected` — True if any method scored a positive
- `detection_method` — winning method (e.g. `guard_pattern:PROMPT_INJECTION`)
- `confidence` — clamped `[0, 1]`
- `cluster_id` — for weak-region tracking (`GUARD_PATTERN_*`, `MANIFOLD_*`, `SUPPLY_CHAIN_*`, ...)
- `fix_proposal` — remediation template if detected, gap documentation if missed

`BlueReport.signature` is an HMAC over the sorted payload.

CLI

```
python axiom_blue_agent.py
```

Pairs with RedAgent automatically: imports `RedAgent`, runs all 8 vectors, then runs all 5 detection methods against each, and prints a DETECTED / MISSED summary with detection counts and report HMAC.